# SIMATS ENGINEERING
# ASSESSMENT TOOL 3: Application Based Questions

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

---

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

---

Code of Conduct
I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Athavan K

**Assessment Tasks**

**Application Based Questions**

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy.
How can encryption and access control protect sensitive backup data?
Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.

3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.

5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

**Application Based Questions Rubric**

| Criteria | Weightage | Level 4 - Exemplary | Level 3 - Proficient | Level 2 - Developing | Level 1 - Beginning |
|---|---|---|---|---|---|
| Relevance to Application | 25% | Response directly addresses the given use case with strong relevance | Mostly relevant response | Partially relevant | Weak relevance |
| Use of Cloud Concepts | 25% | Applies appropriate concepts accurately | Mostly accurate application | Basic application only | Incorrect concept usage |
| Practical Insight | 20% | Shows strong real-world understanding | Shows reasonable practical understanding | Limited practical insight | No practical insight |
| Completeness | 15% | Response covers all required aspects | Covers most aspects | Covers only basic aspects | Incomplete response |
| Clarity | 15% | Clear and concise answer | Generally clear | Some confusion in expression | Poor clarity |

## Question 1 — Auto-Scaling and Load Balancing for Seasonal Traffic Spikes

### Scenario

A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing handle this demand, how this reduces downtime and improves user experience, the role of monitoring tools, and relate this to a real-world e-commerce scenario.

### 1. Handling Demand with Auto-Scaling and Load Balancing

Auto-scaling automatically adjusts the number of active compute instances based on real-time demand signals such as CPU utilization, request rate, or queue depth. During a sale event, as traffic climbs, the auto-scaling group provisions additional instances within minutes; once the surge subsides, it scales back down, so the company only pays for capacity it actually uses. A load balancer sits in front of this fleet and distributes incoming requests evenly across all healthy instances, using health checks to automatically stop routing traffic to any instance that becomes unresponsive.

### 2. Reducing Downtime and Improving User Experience

Because capacity grows in step with demand, the application avoids the resource exhaustion (CPU/memory saturation, connection limits) that causes slow page loads, failed checkouts, or full outages during flash sales. The load balancer's health checks mean a single failing instance is transparently removed from rotation without customers noticing, so the site remains available and responsive — directly translating to fewer abandoned carts and higher conversion during peak shopping windows.

### 3. Role of Monitoring Tools

- **Metrics-driven scaling:** Provide the real-time metrics (CPU, memory, request latency, error rate) that trigger auto-scaling policies in the first place.
- **Proactive alerting:** Alert operations teams proactively when thresholds are approached, allowing manual intervention before automated scaling limits are reached.
- **Bottleneck visibility:** Give visibility into which components are bottlenecks (e.g., database vs. web tier), guiding where additional scaling or optimization is needed.
- **Capacity planning:** Support post-event capacity planning by analyzing traffic patterns from the sale to better provision for the next one.

### 4. Real-World E-Commerce Example

A retailer such as Amazon or Flipkart during a Black Friday or Big Billion Days sale sees traffic jump many times over normal levels within minutes of the sale opening. Cloud auto-scaling provisions hundreds of additional web-tier and application-tier instances automatically, an elastic load balancer spreads checkout and browsing requests across them, and monitoring dashboards let the operations team watch database and paymentgateway latency in real time so they can intervene if a downstream dependency becomes the bottleneck — keeping the site available throughout the event.

### Justification

Auto-scaling and load balancing together directly address unpredictable demand by matching capacity to load dynamically rather than relying on fixed over-provisioning, while monitoring closes the loop by supplying the data that drives scaling decisions and by surfacing problems before they affect customers.

**Question 2 )Secure Cloud Backup Storage for an Organization Scenario**

An organization wants to store large volumes of backup data securely in the cloud. Discuss durability and redundancy, how encryption and access control protect sensitive data, cost advantages over traditional storage, and give a disaster recovery example.

**1. Durability and Data Redundancy**

Cloud storage services achieve extremely high durability (often quoted at 99.999999999%, i.e. "11 nines") by automatically replicating every object across multiple physical devices and, typically, multiple availability zones within a region. If a disk, server rack, or even an entire data center fails, the data remains intact and accessible from the surviving replicas. Many providers also offer cross-region replication so a full copy of backup data exists in a geographically distant location, protecting against regional disasters.

**2. Encryption and Access Control**

- **Encryption at rest:** Data at rest is encrypted (commonly AES-256) automatically by the storage service, with encryption keys managed either by the provider or by the organization via a dedicated key-management service for stricter control.
- **Encryption in transit:** Data in transit between the organization and the cloud is protected using TLS, preventing interception during upload or restore.
- **Access control (IAM):** Identity and Access Management (IAM) policies enforce role-based, least-privilege access so only authorized personnel or services can read, write, or delete backups.
- **Delete protection:** Multi-factor authentication and delete-protection (e.g., MFA-delete, object lock/immutability) prevent accidental or malicious deletion of backup data, including protection against ransomware overwriting backups.

**3. Cost Advantages Compared to Traditional Storage**

Traditional on-premises backup requires large upfront capital expenditure on storage hardware, off-site tape rotation logistics, and ongoing maintenance staff. Cloud storage instead uses a pay-as-you-go model, and tiered storage classes (hot, cool/infrequent-access, archive) let the organization automatically move older backups to much cheaper archival tiers, paying only for the capacity actually consumed rather than provisioning for worst-case future growth.

**4. Disaster Recovery Use Case**

Consider a company whose on-premises data center suffers a fire or flood. Because nightly backups were continuously replicated to cloud storage in a separate region, the organization can spin up replacement infrastructure in the cloud (or a secondary site) and restore databases and file systems from the most recent cloud backup, resuming operations within hours instead of losing the data permanently or waiting weeks for on-premises hardware to be replaced.

**Justification**

Built-in replication delivers the durability and redundancy the scenario requires without manual effort, layered encryption and IAM protect sensitive backups from both external and insider threats, and tiered pricing makes long-term retention economically viable — collectively enabling a reliable, low-cost disaster-recovery capability.

**Question 3 ) Global Deployment of a Mobile App Using CDNs Scenario**

A mobile app company plans to deploy its application globally. Explain how CDNs improve performance and reduce latency, how edge servers enhance user experience across regions, how cloud providers support global availability, and relate this to a real-time streaming or gaming application.

### 1. How CDNs Improve Performance and Reduce Latency

A Content Delivery Network caches static and semi-static content (images, videos, app assets, API responses that change infrequently) on servers distributed across many geographic locations worldwide. When a user requests content, it is served from the nearest CDN edge location rather than traveling all the way back to a single origin server, which significantly cuts round-trip time and reduces load on the origin infrastructure.

### 2. Role of Edge Servers Across Regions

- **Proximity:** Serve cached content from a location physically close to the user, minimizing network hops and propagation delay.
- **Load absorption:** Absorb traffic spikes locally, so a surge of users in one region does not overwhelm the origin server or degrade service for users elsewhere.
- **Edge compute:** Can perform edge computing tasks (authentication checks, request routing, image resizing) closer to the user, further reducing perceived latency.
- **Resilience:** Provide built-in resilience — if one edge location fails, traffic is automatically redirected to the next-nearest healthy edge node.

### 3. How Cloud Providers Support Global Availability

Major cloud providers operate a global network of regions and availability zones along with globally distributed CDN edge points. They offer global load-balancing/traffic-management services that route each user to the nearest healthy region, multi-region database replication to keep data consistent worldwide, and global backbone networks that carry traffic between regions faster and more reliably than the public internet — allowing the mobile app company to deploy once and reach users on every continent with consistent performance.

### 4. Real-Time Streaming / Gaming Example

A live-streaming platform such as a global video-streaming or mobile gaming service uses CDN edge servers to cache video segments and game assets close to players in each region, while edge compute nodes handle latency-sensitive tasks like matchmaking or chat locally. This is why a player in Asia and a player in Europe can both experience low-latency, smooth gameplay or streaming simultaneously — the CDN and edge infrastructure prevent every request from having to cross the globe to a single origin server.

**Justification**

Caching content at globally distributed edge locations directly reduces the physical distance data must travel, which is the primary driver of latency; combined with cloud providers' multi-region infrastructure, this gives the mobile app company the consistent, low-latency global availability that real-time applications like streaming and gaming require.

## Question 4 ) Regulatory Compliance and Security for a Financial Institution Scenario

A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing, the role of logging, monitoring, and encryption in security, how data sovereignty requirements can be met, and give an example involving sensitive financial transactions.

### 1. Cloud Provider Support for Compliance and Auditing

Major cloud providers maintain certifications against financial and data-protection standards (e.g., PCI-DSS, SOC 2, ISO 27001, and region-specific regulations) and publish compliance reports the institution can use as part of its own audit evidence. They offer dedicated compliance and audit-management services that continuously assess configurations against regulatory benchmarks and flag non-compliant resources automatically, and provide immutable audit trails of every API call and configuration change (e.g., AWS CloudTrail, Azure Monitor).

### 2. Role of Logging, Monitoring, and Encryption in Security

- **Logging:** Every access to sensitive systems and data is recorded in tamper-evident logs, creating an audit trail regulators and internal auditors can review to verify who accessed what and when.
- **Monitoring:** Continuous monitoring with automated alerting detects anomalous behavior (e.g., unusual login locations, privilege escalation, large data exports) in near real time, enabling rapid incident response.
- **Encryption:** Encryption at rest and in transit (AES-256, TLS 1.2+) ensures that even if data is intercepted or storage media is compromised, financial data remains unreadable without the encryption keys, which are themselves tightly access-controlled via a key-management service.

### 3. Meeting Data Sovereignty Requirements

The institution can select specific cloud regions to ensure customer and transaction data is stored and processed only within the required national or regional boundaries, use region-locking policies enforced through IAM and storage configuration to prevent data from being replicated outside permitted jurisdictions, and rely on the provider's data-residency commitments and compliance documentation to demonstrate adherence to local financial-data regulations during audits.

### 4. Example — Sensitive Financial Transactions

Consider an online banking platform processing customer fund transfers. Every transaction is encrypted end-to-end, access to the transaction database is restricted to specific service accounts under least-privilege IAM roles, all access and changes are logged immutably for regulator review, monitoring tools flag any anomalous transaction pattern suggestive of fraud, and the underlying data is stored only in the cloud region required by the country's financial regulator — together satisfying both security and data-sovereignty obligations.

**Justification**

Provider certifications and automated compliance tooling reduce the institution's audit burden, layered logging/monitoring/encryption directly protects the confidentiality and integrity of financial data, and region-scoped deployment ensures the data-sovereignty requirements central to financial regulation are met.

**Question 5 )Containerization and Orchestration for Application Deployment**

**Scenario**

A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in performance and resource usage, how orchestration (like Kubernetes) simplifies deployment and scaling, portability benefits across cloud environments, and give a microservices example.

### 1. Containers vs. Virtual Machines

A virtual machine virtualizes an entire hardware stack and runs a full guest operating system on top of a hypervisor, making each VM relatively heavyweight (gigabytes in size) and slow to boot (minutes). A container, by contrast, virtualizes only at the operating-system level, sharing the host kernel while isolating the application and its dependencies in a lightweight, portable package (typically megabytes) that starts in seconds. This means far more containers than VMs can run on the same physical hardware, with lower memory and CPU overhead and much faster startup, at the cost of slightly weaker isolation than a full VM.

### 2. How Orchestration (Kubernetes) Simplifies Deployment and Scaling

- **Scheduling:** Automates scheduling of containers onto available nodes based on defined resource requirements, without manual placement decisions.
- **Self-healing:** Automatically restarts failed containers and reschedules them onto healthy nodes, providing self-healing without operator intervention.
- **Auto-scaling:** Scales the number of container replicas up or down automatically based on CPU/memory or custom metrics via the Horizontal Pod Autoscaler.
- **Rolling deployments:** Manages rolling updates and rollbacks, allowing new application versions to be deployed with zero downtime and reverted instantly if a problem is detected.
- **Service discovery:** Provides built-in service discovery and internal load balancing so containers can find and communicate with each other reliably as they scale.

### 3. Portability Benefits Across Cloud Environments

Because a container packages the application together with all its dependencies and runs on any system with a compatible container runtime, the same container image behaves identically whether it runs on a developer's laptop, an on-premises server, or any major cloud provider's Kubernetes service (EKS, AKS, GKE). This eliminates the "it works on my machine" problem, allows the team to avoid vendor lock-in, and makes multi-cloud or hybrid-cloud deployment straightforward since the orchestration layer itself is largely cloud-agnostic.

### 4. Microservices Example

An e-commerce application built as microservices might split functionality into separate containerized services — a product catalog service, a shopping cart service, a payment service, and a recommendation engine — each independently deployable, scalable, and updatable. Kubernetes orchestrates all of these, scaling the payment service independently during checkout-heavy periods and the catalog service independently during browsing-heavy periods, while service discovery lets them communicate reliably as instances are added or removed.

### Justification

Containers' lightweight, OS-level virtualization directly reduces resource overhead and startup time compared to VMs, Kubernetes automates the operational complexity of running many containers reliably at scale, and the resulting portability lets the development team deploy the same microservices-based application consistently across any cloud environment without rework.